Qwen

An Invariant-Driven Architecture for Systemic Horror Audio: A Framework for Hellscape Mixer and Binaural Field Generation

This report details a comprehensive research framework for developing the audio backbone of Horror$Place, centered on an invariant-driven paradigm. It establishes a systematic approach where all ambient, pressure, and spectral bed decisions originate from core game-world states—termed invariants—and player engagement metrics. This foundation prioritizes emergent soundscapes over discrete event triggers, creating a more coherent and contextually rich horror experience. The framework emphasizes the encapsulation of complex mathematical models behind intuitive, qualitative contract schemas and Lua policy surfaces, enabling both human designers and AI systems to reason about and manipulate the audio system effectively. Furthermore, it introduces a robust, empirical validation loop using telemetry and hex-coded mapping families, allowing the audio engine to evolve based on data-driven performance rather than subjective playtesting alone. The entire architecture is designed for seamless integration across a multi-repository constellation, ensuring consistency and scalability while maintaining strict safety and governance protocols. This document synthesizes the provided materials to present a blueprint for a novel and powerful methodology in interactive media sound design.

The Core Paradigm: From Event Triggers to Emergent Soundscapes

The central principle underpinning the proposed audio pipeline for Horror$Place represents a fundamental philosophical shift away from traditional, event-driven sound design towards a systemic, invariant-driven paradigm . In conventional game audio, sounds are often treated as discrete artifacts tied to specific events; for example, a monster appearing might trigger the playback of a specific "scream" sound file. While effective for clear-cut actions, this approach struggles to create a truly immersive and

believable world, as the soundscape can feel static or disconnected from the environment's deeper history and dynamic state. The new framework rejects this model as a primary mechanism. Instead, it posits that the default substrate for the Hellscape Mixer—the system responsible for generating the ambient soundscape—must be rooted in the continuous flow of information derived from the game world's invariants and entertainment metrics . All decisions regarding ambient textures, atmospheric pressure, and background spectral activity must originate from these core data points before any higher-level, event-driven patterns are considered .

The invariants form the bedrock of this system, providing a quantitative description of the game world's geospatial and historical properties. These include Catastrophic Imprint (CIC), which measures the intensity of traumatic events at a location; Moral Decay Index (MDI); Archival Opacity/Sparseness (AOS), which describes how obscured or muffled information is; Ritual Residue Measure (RRM), indicating the presence of recurring, patterned phenomena; Haunted Vector Field (HVF), which provides directional cues for presences; Liminal Stress Gradient (LSG), defining stress levels at transitional spaces; Spectral-History Coupling Index (SHCI), controlling the literalness of spectral cues; and Dread Exposure Threshold (DET), which acts as a master cap on intensity to prevent overwhelming the player . Entertainment metrics such as the Uncertainty Engagement Coefficient (UEC), Suspense-Tension Contrast Index (STCI), Confusion-Dissociation Level (CDL), and Ambiguous Resolution Ratio (ARR) track the player's cognitive state and engagement level . By making these values the primary inputs for audio generation, the system ensures that the sonic environment is always reflective of the player's immediate context. For instance, the same door-opening event in a high-CIC war ruin would produce a different, more historically resonant and potentially louder or more distorted sound than the same action in a low-CIC, neutral office corridor . This prevents designers from bypassing the world's established lore and creates a consistent, believable atmosphere where the environment itself feels imbued with history and dread . The explicit goal is to make every decision flow through a "QueryHistoryLayer → SetBehaviorFromInvariants → DriveEngineAudio" pipeline, rather than relying on ad-hoc event triggers .

Event-triggered sounds are not eliminated but are instead re-contextualized as a constrained, higher-level layer operating *upon* the invariant-driven base . These events, such as a jump scare or a combat impact, no longer directly play a sound file. Instead, they request a specific delta or motif from the Hellscape Mixer, which then determines how to express that request within the boundaries set by the local invariants and metrics . For example, a monster spawn event might signal a desire for a loud, impactful sound. The Hellscape Mixer would query the local CIC, LSG, and DET. If these values are high, it might generate a deep, resonant boom that aligns with the traumatized environment. If the DET is already high, it would scale back the intensity of that boom to stay within the

safety envelope. If the STCI is high (indicating a need for suspense), it might choose a sudden silence followed by a subtle whisper rather than a loud roar. This approach maintains the power of event-driven moments while grounding them in the world's logic, preventing auditory jarring and preserving immersion. The system becomes one of contextual expression rather than simple reaction.

This paradigm shift has profound implications for quality and coherence. By treating audio as an emergent property of the world state, the system inherently avoids common pitfalls like flat corridors, lifeless firefights, or mismatched environmental sounds, which are often symptoms of poor event design . A "flat corridor" failure, for instance, would be identified as a tile with low CIC and LSG but marked as a liminal space, causing the audio to lack the necessary threshold effects . This failure can be codified into linters and mood contracts that prevent such inconsistencies from being merged into the main codebase . Similarly, a "lifeless firefight" is diagnosed as a battle zone with low CIC/HVF or low DET, resulting in sterile audio without catastrophe echoes or directional pressure . The invariant-driven model makes these failures obvious because the required conditions for a rich, horrific soundscape are simply not met. This transforms audio design from a series of manual placements into the creation of a robust, rule-based system that consistently generates appropriate content based on a shared vocabulary of invariants and metrics . The ultimate goal is to achieve a soundscape so deeply integrated with the world that players perceive it not as a collection of sounds, but as an intrinsic part of the haunting itself.

Architectural Blueprint: A Multi-Layered System for Systemic Audio

The invariant-driven audio framework for Horror$Place is not a single monolithic component but a sophisticated, multi-layered architectural system designed for modularity, specialization, and scalability. This blueprint separates concerns into distinct layers, each with a specific role in translating raw world-state data into a compelling and safe auditory experience. This modular design allows different teams and repositories to work on specific aspects of the pipeline—invariants, policies, empirical research, and engine implementation—without introducing conflicts or breaking the overall system integrity . The architecture can be conceptualized as four primary layers: the Foundation Layer, the Policy Layer, the Empirical Layer, and the Adapter Layer.

The **Foundation Layer** constitutes the ground truth of the system. It is composed of two key components: the invariant definitions and the entertainment metrics. The invariants —such as CIC, MDI, AOS, HVF, LSG, SHCI, and DET—are quantifiable properties of the game world that describe its history and spatial dynamics . They provide the raw, objective data about where the player is and what kind of place it is. The entertainment metrics—UEC, EMD, STCI, CDL, and ARR—track the player's subjective experience and cognitive state during gameplay . They provide the objective data about how engaged, uncertain, confused, or resolved the player is feeling. Together, these two sets of data form the complete input vector for the entire audio engine. This layer is implemented primarily in Tier 1/Tier 2 repositories like `Horror.Place` and is exposed to the rest of the system through a canonical, standardized API, such as `H.sampleAll(regionId, tileId, playerId)` . This ensures that every component interacting with the audio system accesses a consistent and authoritative source of truth.

The **Policy Layer** acts as the brain of the operation, interpreting the data from the Foundation Layer and translating it into abstract audio behaviors. This is the heart of the invariant-driven logic. Implemented in Lua modules like `enginelibrary/horroraudio.lua` and `binaural_field.lua`, this layer contains the rules and mappings that define how invariants and metrics influence audio dimensions like density, clarity, directionality, and repetition . The Lua policy queries the invariants and metrics via the `H.` API and computes a set of abstract descriptors. These descriptors are not concrete sound files or direct engine calls; instead, they are pure data tables containing tags, weights for different sound families, and target values for a standardized set of Real-Time Parameter Controls (RTPCs) . This abstraction is critical, as it creates a thin, engine-agnostic policy layer that decouples the high-level audio logic from the specifics of any particular game engine or middleware like Wwise or FMOD . The policy module always respects the safety caps defined by the DET metric before emitting any output, ensuring the audio remains within a comfortable and engaging range for the player .

The **Empirical Layer** introduces a scientific method for evolving the audio engine beyond static rules. This layer is housed in Tier 3 research-oriented repositories like `Neural-Resonance-Lab` and `Redacted-Chronicles` . Its purpose is to test, measure, and refine the mappings defined in the Policy Layer. It operates by instrumenting the system to collect detailed telemetry on every audio decision and its subsequent effect on player engagement metrics . This data is used to evaluate the performance of different mapping strategies, often represented as hex-coded "mapping families" (e.g., linear, sigmoid, hysteresis curves) . Through controlled experiments, researchers can empirically determine which mathematical approaches to mapping invariants to RTPCs reliably produce desired outcomes, such as raising UEC and ARR without violating DET or CDL caps . Once a mapping family proves its effectiveness, it is promoted back into the core

logic of the Policy Layer, effectively upgrading the deterministic baseline with empirically validated improvements . This creates a powerful feedback loop where the "deterministic" system is continuously refined by data-driven research, balancing stability with innovation.

The **Adapter Layer** serves as the bridge, translating the abstract audio descriptors from the Policy Layer into concrete commands for the game engine. This layer is typically implemented in C++ or another engine-native language and consists of components like `UHorrorBinauralFieldComponent` in Unreal Engine . These adapters call the Lua policy functions once per frame or at a suitable cadence, receive the pure data table in return, and then execute engine-specific operations . For example, it might map a `base.intensity` value from the Lua table to a Wwise RTPC named `bf_intensity` or use `orbits` data to position and control one or more `UAudioComponent` instances in 3D space . This layer is kept intentionally thin and dumb; its sole responsibility is marshalling data and calling engine APIs. It does not contain any complex logic or decision-making. The stability of the entire system relies on the predictability of this adapter, which consumes a stable, well-defined interface from the Lua policy layer. This separation of concerns ensures that the complex audio logic remains portable and can be adapted to different engines with minimal changes to the core policy, while the engine-specific implementation details are cleanly isolated in the adapter layer . This four-layer architecture provides a robust, scalable, and scientifically-grounded framework for creating a deeply integrated and responsive audio experience.

Encapsulation Strategy: Mathematical Formulations Behind Qualitative Contracts

A cornerstone of the research framework for Horror$Place's audio pipeline is a deliberate and disciplined strategy of encapsulation. The system acknowledges the necessity of concrete mathematical formulations to precisely map invariants to audio dimensions but mandates that these complex formulas remain hidden from public-facing interfaces . Instead, the official, human-readable, and AI-reasonable specifications—such as JSON schemas and Lua policy surfaces—expose only qualitative concepts, bounded numeric ranges, and safety envelopes . This approach mirrors the successful implementation seen in the BCI-driven audio research, where intricate curve families and parameter sets exist as lab-only assets, while external systems interact with them through stable, high-level abstractions . This strategy achieves a crucial balance: it provides researchers and engineers with the freedom to iterate on the underlying physics of fear through rigorous

mathematical modeling, while simultaneously offering designers, writers, and automated tools a stable, understandable, and auditable interface for building and validating the audio experience .

The first step in this strategy is the formal definition of qualitative contract schemas. These schemas serve as the public contracts that govern audio behavior across the system. One of the most critical is the `audiolandscape_style_v1` schema, which maps bands of invariant values to abstract sound families and defines their intended metric impacts . For example, a style contract could specify that when CIC is between 0.4 and 0.7, LSG is between 0.6 and 1.0, and AOS is between 0.4 and 0.8, the system should apply the `Trapped_Phantom` preset, which is intended to create localized entity loops and maintain an ARR greater than or equal to 0.7 while keeping CDL below 0.6 . This contract exposes a clear intent—"create localized entity loops"—and a set of quantitative targets ("ARR $\geqslant$ 0.7", "CDL $\leqslant$ 0.6") without revealing the specific mathematical formula used to generate the audio . Another key contract type is the `mood_contract`, which defines acceptable invariant ranges and target metric windows for specific gameplay modes, such as `DreadForgeResonance.Audio` . These contracts are validated by Continuous Integration (CI) systems to ensure compliance, preventing configurations that violate established safety or engagement principles .

Within the internal research and engine libraries (Tier 1/Tier 2), these qualitative contracts are brought to life by concrete mathematical formulations. The actual mapping from invariants to audio dimensions is performed by small, dedicated formulas or curve functions residing in Lua libraries or Rust modules . For instance, the concept of `audio_clarity` mentioned in passing could be formally defined by a function like $Clarity = 1 - (\beta_{AOS} \cdot AOS)$, where β_{AOS} is a tunable constant . Similarly, the density of the soundscape could be modeled as a weighted sum of multiple invariants, such as $Density = \alpha_{CIC} \cdot CIC + \alpha_{MDI} \cdot MDI$, where the coefficients α represent the sensitivity of density to each invariant . These mathematical models are not arbitrary; they are the subject of empirical research. Different curve families, such as linear ($y = ax + b$), sigmoidal, piecewise, or hysteresis-based functions, can be tested to see which shape produces the most desirable effect on player metrics like UEC and ARR . This research is conducted in Tier 3 labs, where hex-coded identifiers (e.g., `0xHKLIN` for linear, `0xHKSIG` for sigmoid) are used to tag and manage different versions of these mapping functions .

This encapsulation provides several significant advantages. First, it future-proofs the public contracts. If a researcher discovers a superior mathematical model for mapping CIC to pressure—for instance, a sigmoidal curve instead of a linear one—the change can be made internally in the library without requiring any updates to the

`audiolandscape_style_v1` schema or any of the audio assets that reference it . The qualitative intent and quantitative bounds remain the same, preserving stability for the broader project. Second, it enables AI-powered co-design. An AI chat tool can be trained to understand the qualitative contracts and the available invariant/metric vocabulary. When a designer asks for "more ambiguity," the AI can reason about the contract and suggest modifying parameters to increase AOS or decrease ARR within safe bounds, rather than inventing its own bespoke audio calls that would break the system . Third, it facilitates better governance and safety. The CI system can easily verify that a given contract's numeric ranges fall within predefined, safe limits. However, if the underlying math were exposed, it would be far more difficult to audit for potential edge cases or runaway behaviors. By keeping the math internal, the safety logic can be centralized and rigorously enforced by a trusted runtime environment, such as the Rust mapping engine, which is responsible for applying DET-aware caps and velocity limits to all outputs before they are passed to the engine adapter . This layered approach of qualitative contracts governing quantitative implementations, which are themselves guided by rigorous mathematical models, is the key to building a system that is both flexible enough for creative exploration and stable enough for large-scale production.

Cross-Repository Implementation: A Cohesive Constellation of Audio Logic

The true strength of the proposed audio framework lies in its explicit design for a multi-repository, collaborative development environment. Rather than being confined to a single codebase, the system is structured as a constellation of interconnected repositories, each with a clearly defined role in the audio pipeline . This division of labor is essential for managing complexity, enabling parallel development, and ensuring that all components adhere to a single, consistent set of principles. Repositories like `Horror.Place` for core logic, `Spectral-Foundry` for style and persona contracts, `Atrocity-Seeds` for invariant-tagged SFX palettes, and `Neural-Resonance-Lab` for empirical research all contribute to a unified whole, preventing the fragmentation of behavior that can plague large-scale projects . This cohesive implementation is built upon shared schemas, standardized APIs, and a clear data flow between tiers.

The core logic and foundational specifications reside primarily in Tier 1/Tier 2 repositories, with `Horror.Place` serving as the central hub . Here, the fundamental building blocks are defined. This includes the JSON schemas for invariants (`invariantsv1.schema.json`) and entertainment metrics

(`entertainmentmetricsv1.schema.json`), which establish the universal vocabulary of the system . It also houses the definitive audio contracts, such as `moodcontractv1` and `audiolandscapestylev1`, which codify the rules for how audio should behave under specific conditions . The canonical Lua APIs, found in modules like `horrorinvariants.lua` and `horroraudio.lua`, provide the standardized entry points for querying the world state and composing audio behaviors . Finally, the C++ or engine-specific adapter code, such as the `HorrorAudioDirector` class or `UHorrorBinauralFieldComponent` in Unreal, lives here, acting as the thin bridge that translates the abstract Lua descriptors into concrete engine calls . Any change to these core elements requires careful review and testing to ensure backward compatibility and adherence to the established contracts.

On top of this core foundation, Tier 2 repositories like `Spectral-Foundry` build out the creative and experiential layers. This repository is responsible for defining higher-level concepts like "audio landscape styles" and "persona hooks" . Style contracts in `Spectral-Foundry` link the abstract invariant bands from `audiolandscapestylev1` to concrete audio buses, RTPC mappings, and even bus layouts in middleware like Wwise or MetaSounds . Persona contracts define how different AI personas, such as the "Archivist" or "Professor," have distinct preferences for audio mixing. These personas can be programmed to bias toward certain mapping profiles or audio styles based on their learned goals, which are updated according to telemetry showing which configurations lead to better engagement metrics . For example, an Archivist persona might favor audio profiles that maximize ARR and EMD in high-AOS regions, reflecting its goal of uncovering hidden truths . This layer allows for the creation of varied and personalized experiences without altering the underlying invariant-to-audio logic.

The asset pipelines are handled by specialized repositories like `Atrocity-Seeds` and `Black-Archivum`. These repositories do not just store sound files; they house invariant-tagged SFX palettes and region packs . Each sound grain or SFX palette is tagged with the specific invariant bands it is designed to respond to. For example, a "warruin_highCIC_highAOS" palette would be loaded in areas with high catastrophic imprint and archival opacity . The NDJSON invariant packs stored here explicitly attach CIC, MDI, AOS, HVF, and LSG values to region and tile IDs, providing the raw data that the Hellscape Mixer uses to select appropriate SFX grains dynamically . This tight coupling between assets and the invariants they are meant to represent is what allows the Hellscape Mixer to function as a true "default" audio runtime, capable of generating a plausible and thematically consistent soundscape even in graybox maps or areas with no bespoke audio art .

Finally, the evolution and refinement of the audio system occur in Tier 3 repositories, namely `Neural-Resonance-Lab` and `Redacted-Chronicles` . This is the research and experimentation layer. It contains the telemetry schemas for logging every audio decision, including the full snapshot of invariants and metrics, the chosen mapping family (identified by its hex code), the resulting RTPC outputs, and the subsequent deltas in player engagement metrics (ΔUEC, ΔARR, etc.) . This rich dataset is fed into Kotlin analysis tools that rank different mapping families based on their performance, driving an evolutionary loop where only the most effective profiles are promoted back to the Tier 1/ Tier 2 repositories . This lab environment also explores advanced concepts like BCI-driven audio, where derived fields like `bcifearindex` and `bciattentionfocus` are used as additional inputs to the mapping functions, further refining the system's ability to adapt to the player's real-time physiological state . By clearly delineating these responsibilities, the entire constellation works in harmony, with each part contributing to a single, coherent, and ever-improving audio backbone for Horror$Place .

Repository	Tier	Primary Role	Key Artifacts
`Horror.Place`	1	Core Logic & API	Invariant/Metric Schemas (`*.schema.json`), Lua APIs (`horrorinvariants.lua`, `horroraudio.lua`), C++ Adapters (`HorrorAudioDirector`), Mood Contracts (`moodcontractv1`).
`Spectral-Foundry`	2	Style & Persona Contracts	Audio Landscape Styles (`audiolandscapestylev1`), Persona Hooks (e.g., `audiopressurebias`), Bus/RTPC Mappings for Middleware.
`Atrocity-Seeds` / `Black-Archivum`	2	Invariant-Tagged Assets	Invariant Packs (`regioninvariantpack.*.ndjson`), SFX Palettes (tagged by archetype/invariant bands), Tileset Contracts.
`Neural-Resonance-Lab` / `Redacted-Chronicles`	3	Empirical Research & Evolution	Telemetry Schemas (`audio_rtpc_mapping_telemetry_v1.ndjson`), Hex-Coded Mapping Families (e.g., `0xPKLIN`), Kotlin Analysis Tools, BCI Input Schemas.

Empirical Evolution: Validating Mappings Through Telemetry and Hex-Coded Families

The invariant-driven audio framework for Horror$Place is designed not as a static system of rules, but as a dynamic and evolvable engine. Central to this evolution is a rigorous empirical validation process that moves beyond theoretical design and subjective evaluation, grounding audio tuning in measurable player data. This process is facilitated by a combination of detailed telemetry logging and a systematic approach to testing different mathematical mappings, represented by hex-coded "mapping families." This methodology, originating from the BCI audio research, allows the team to treat audio

design as a scientific experiment, constantly testing hypotheses, measuring outcomes, and promoting only those configurations that demonstrably improve engagement while respecting strict safety constraints . This creates a powerful feedback loop where the deterministic baseline of the audio system is periodically upgraded with empirically proven enhancements .

The foundation of this empirical process is a comprehensive telemetry system that meticulously logs every audio decision and its consequences. A dedicated schema, tentatively named `schemas/audio_rtpc_mapping_telemetry_v1.json`, defines the structure for this data . Per-decision records are generated, capturing a rich snapshot of the system's state. This includes the precise values of all relevant invariants (CIC, AOS, LSG, HVF, DET) and entertainment metrics (UEC, ARR, CDL) at the moment of the decision . Crucially, the log must also record the identity of the mapping family that was executed, identified by its unique hex code (e.g., `0xPKLIN` for a linear mapping) and any associated variant ID . The next key piece of data is the complete vector of RTPC outputs generated by that mapping family. Finally, and most importantly, the telemetry captures the outcome by logging the subsequent deltas in player engagement metrics over a short time window (e.g., 5-20 seconds): ΔUEC, ΔARR, ΔCDL, ΔEMD, and ΔSTCI . This dataset effectively creates a "proof" for each audio decision, linking a specific input state and a specific mapping strategy to its measurable effect on the player's experience. This data is invaluable for identifying correlations, such as whether a certain mapping family consistently raises UEC in high-CIC zones without causing a corresponding spike in CDL.

With this telemetry infrastructure in place, the system can systematically explore the vast space of possible invariant-to-RTPC mappings. This is achieved through the use of hex-coded mapping families, which act as pluggable "genes" for the audio engine . Researchers can define and test different curve shapes and logical structures. For example, a linear family (`0xPKLIN`) might use a simple formula like $RTPC_{out}=a\cdot Input+b$. A sigmoidal family (`0xPKSIG`) would use a logistic function, $RTPC_{out}=\frac{1}{1+e^{-k(Input-c)}}$, which is useful for creating sharp transitions at thresholds . Other families could implement more complex behaviors like hysteresis, where the mapping curve differs for rising versus falling input values, or oscillatory functions that introduce rhythmic pulsing, perhaps phase-locked to a BCI-derived `bcibreathphase` signal . Each of these families would have its own parameter vector (the `a`, `b`, `k`, `c` values) that can be mutated and tested.

The evolution process follows a clear cycle. Controlled experiments are run, often rotating different mapping families in identical scenes while holding player and environmental conditions as constant as possible. The resulting telemetry is then analyzed by Kotlin-based tools in the `Neural-Resonance-Lab` repository . These tools

aggregate the data, compute statistics like mean and variance for the metric deltas (e.g., $\mathbb{E}[\Delta\text{UEC}]$, $\mathbb{E}[\Delta\text{ARR}]$), and calculate rates of negative outcomes like overload or dropout . A composite ranking score can be calculated for each mapping profile, for instance:

$$Score = w_U \cdot \mathbb{E}[\Delta\text{UEC}] + w_A \cdot \mathbb{E}[\Delta\text{ARR}] - w_O \cdot \Pr(\text{overload}) - w_C \cdot \max(0, \mathbb{E}[\text{CDL}] - \text{CDL}_{\max})$$

where weights (w_U, w_A, w_O, w_C) reflect the relative importance of each factor . Profiles with the highest scores, indicating reliable uplift in positive metrics without breaching safety constraints, are deemed "successful." These successful profiles are then promoted from the Tier 3 experimental pool back into the Tier 1/Tier 2 deterministic set, becoming the new standard for that mapping. This transforms the audio system from a fixed specification into a living, breathing entity that adapts and improves over time based on hard evidence of what creates engaging and effective horror. This scientific approach provides a robust, data-driven path for the continuous evolution of the audio engine.

Spatial Audio Integration: Structuring Binaural and 3D Rhythms

The invariant-driven framework extends seamlessly into the realm of spatial and binaural audio, treating the movement and positioning of sound as another dimension to be governed by the same principles of contract and policy. Instead of being an afterthought or a collection of engine-specific "magic," spatial audio behavior is defined by abstract, engine-agnostic instructions that are generated by the Lua policy layer . This approach ensures that binaural motion, stereo panning, and 3D rhythmic patterns are not arbitrary but are direct, logical responses to the player's location, the geometry of the space, and their current psychological state, as captured by the invariants and metrics . The system provides a clear separation between the definition of spatial behaviors in high-level contracts and their realization in engine-specific adapters, mirroring the overall architecture of the audio pipeline.

The definition of spatial and binaural behavior begins with qualitative contract schemas. For a feature like a binaural orbit field, a Tier 1 contract would define the conditions under which such an effect is permissible . For example, it would declare that a "head-internal" or highly unusual spatial cue is gated by a high SHCI value, preventing impossible audio from occurring in neutral spaces . The contract would also define the metric intent: "modestly increase UEC and EMD; keep ARR ⩾ 0.8; never raise CDL above

0.6" . This contract does not contain any code or raw audio parameters; it merely establishes the rules and goals that any implementing system must follow . The actual mapping from invariants to spatial parameters is handled by a dedicated Lua policy module, such as `enginelibrary/binaural_field.lua` . This module's job is to read the invariants (CIC, LSG, HVF, SHCI) and metrics (UEC, ARR, DET) and translate them into an abstract "orbit plan" . The provided skeleton code demonstrates this perfectly, computing an azimuth, elevation, radius, and speed for an orbit based on the magnitude and direction of the HVF, the level of trauma (CIC), and the degree of coupling (SHCI) .

This abstract orbit plan is returned as a pure data table to the engine adapter, completely devoid of engine-specific calls . The table might look something like this:

```
{
    enabled = true,
    familyId = "BF.ORBIT.DEFAULT.v1",
    base = { intensity = 0.8, pressureLF = 0.4, stereoWidth = 0.7 },
    orbits = {
        {
            role = "primary_whisper",
            azimuthDeg = 90.0,
            elevationDeg = 5.0,
            radiusNorm = 0.3,
            orbitSpeedDeg = 60.0,
            clarity = 0.9,
            weight = 1.0
        }
    }
}
```

This table is the final output of the Horror$Place audio logic. It is self-contained, human-readable, and machine-parsable. It tells the engine adapter exactly what to do in a generic way . The `base` table contains global parameters like `intensity`, `pressureLF`, and `stereoWidth`, which are mapped to general-purpose RTPCs . The `orbits` array contains a list of moving sound sources, each with semantic tags (`role`), spatial coordinates, and behavioral parameters like `clarity` and `weight` . This structure allows for immense flexibility. The engine adapter can interpret this data in various ways depending on the target platform. In Unreal Engine, as shown in the provided pseudocode, this table would be consumed by a `UHorrorBinauralFieldComponent` . The component would map the `base` parameters to Wwise RTPCs and create `UAudioComponent` instances for each orbit,

updating their 3D positions each frame based on the computed `azimuthDeg`, `elevationDeg`, and `radiusNorm` . The `clarity` and `weight` values would drive separate RTPCs to control filters and volume for those components . The entire process—from querying invariants in Lua to positioning virtual audio sources in Unreal—is driven by the abstract data returned by the policy layer, ensuring that the complex logic of binaural horror is always grounded in the game's core state variables.

This engine-agnostic approach is maintained through a standardized vocabulary of abstract RTPCs and parameters. Instead of exposing engine-specific names like `AkSetRTPCValue`, the system defines a clean slate of meaningful names that describe the audio dimension being controlled. Examples from the research include `bf_intensity`, `dlpressurelf`, `bf_stereo_width`, `bf_orbit<em>_clarity`, and `bf_orbit</em>_weight` . This vocabulary forms a stable, cross-platform API between the Lua policy and the C++ adapters. The actual implementation of these RTPCs in middleware like Wwise or MetaSounds is a matter of configuration, not code modification. For example, the `bf_stereo_width` RTPC could be mapped to a bus's stereo width control in Wwise, which might use a combination of mid-side processing and decorrelation at high values, or it could be mapped to a parameter in a MetaSound graph that controls a chorus node . Because the Lua policy only ever refers to the abstract name, the underlying implementation can be changed or optimized without affecting the core logic. This separation of policy from realization is the key to building a robust, scalable, and portable system for generating complex spatial audio experiences that are deeply integrated with the game world's narrative and mechanics.

Reference

1. [PDF] Proceedings of the First Workshop on Commonsense Inference in ... https://aclanthology.org/D19-60.pdf
2. [PDF] User Tasks Description: a Retrospective, Recent Contributions and ... https://hal.science/hal-03857822v1/file/RoCHI%202020%20-%20palanque%20-%20Keynote.pdf
3. [PDF] Train-the-Trainer Concept on Research Data Management - Zenodo https://zenodo.org/records/13927614/files/TtT-Concept_RDM_V5-1_EN.pdf?download=1
4. Practical Verification of Smart Contracts using Memory Splitting https://dl.acm.org/doi/10.1145/3689796

5. Common transmission convergence layer specification Amendment 2 https://www.itu.int/epublications/publication/itu-t-g-9804-2-2021-amd-2-2024-08-indication-of-upstream-wavelength-option-modification-of-deactivate_onu-id-message-and-serial_number_onu-message-to6-updates-and-emergency-stop-state-in-appendix-xi
6. [PDF] Open-Source Register Reference For AMD Family 17h Processors ... https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/programmer-references/56255_OSRR.pdf
7. [PDF] European Union Risk Assessment Report https://echa.europa.eu/documents/10162/605c05d5-0ef9-46cf-b5a2-bb8a51ac26e5
8. Sitemap - IEEE Standards Association https://standards.ieee.org/sitemap/
9. [PDF] for Europe - IRIS https://iris.who.int/bitstreams/7107999d-7e53-47aa-90e8-bb1d162ff46e/download
10. Simple Index - Alibaba Cloud https://mirrors.aliyun.com/pypi/simple/
11. [PDF] D900051 - Part 8 - Special Specifications - FINAL - nysdot https://www.dot.ny.gov/main/business-center/designbuildproject51/repository/D900051%20-%20Part%208%20-%20Special%20Specifications%20-%20FINAL%20-%2007082020.pdf
12. Download book PDF - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-322-86773-5.pdf
13. Appl. Sci., Volume 16, Issue 5 (March-1 2026) – 465 articles - MDPI https://www.mdpi.com/2076-3417/16/5
14. [PDF] 4th National Conference on Advances in Engineering and ... - ijsrst https://ijsrst.com/paper/v5i7.pdf
15. Board Meeting Minutes - Infrastructure - Apache Whimsy https://whimsy.apache.org/board/minutes/Infrastructure.html
16. SUSE-CU-2024:6034-1: Security update of containers/apache-tomcat https://lists.suse.com/pipermail/sle-container-updates/2024-December/003938.html
17. [PDF] Information Society Guest Editors: Marcin Paprzycki Matjaz Gams https://www.researchgate.net/profile/Giovanni-Semeraro/publication/220166335_Learning_Customer_Profiles_Using_Unlabelled_Data/links/5429346c0cf26120b7b5aa04/Learning-Customer-Profiles-Using-Unlabelled-Data.pdf
18. Writing Secure Code Vista | PDF | Vulnerability (Computing) - Scribd https://www.scribd.com/document/334174329/Writing-secure-code-Vista
19. Designing complex, interactive, architectural systems with CIAS-DM https://search.proquest.com/openview/6fd58432625805711f10e7910e4d6c69/1?pq-origsite=gscholar&cbl=18750
20. Security checklists for Ethereum smart contract development https://www.sciencedirect.com/science/article/pii/S2096720925000946

21. Materials Data Science Ontology(MDS-Onto): Unifying Domain ... https://www.nature.com/articles/s41597-025-04938-5
22. Dr. Clair Sullivan - End-to-end Gen-AI - LinkedIn https://www.linkedin.com/in/dr-clair-sullivan
23. Oracle Architecture Center https://docs.oracle.com/en/solutions/index.html
24. [PDF] D4.1 - Semantic artefact governance models and disciplinary ... - HAL https://hal.science/hal-04678465v1/file/D4.1_FAIR-IMPACT_Semantic_artefact_governance_models_20240731_v2.docx%20%282%29.pdf
25. Formal Methods at the Crossroads. From Panacea to Foundational ... https://link.springer.com/content/pdf/10.1007/b11934.pdf
26. [PDF] Making Perfect Life - European Parliament https://www.europarl.europa.eu/RegData/etudes/etudes/join/2012/471574/IPOL-JOIN_ET%282012%29471574_EN.pdf
27. 333333 23135851162 the 13151942776 of 12997637966 https://www.cs.princeton.edu/courses/archive/spring18/cos226/assignments/autocomplete/testing/words-333333.txt
28. Google 1-gram - CMU School of Computer Science https://www.cs.cmu.edu/~roni/11761/2017_fall_assignments/hw3_stats_google_1gram.txt
29. Foundry MCP Server - Microsoft Developer Blogs https://devblogs.microsoft.com/foundry/feed/
30. (PDF) AI/ML-Driven Design Verification: Applications, Architecture ... https://www.researchgate.net/publication/393948154_AIML-Driven_Design_Verification_Applications_Architecture_and_Methodology_Evolution
31. DeFi Agents API - AI Agent Definitions for Web3 - LobeHub https://lobehub.com/de/mcp/nirholas-defi-agents
32. 333333 23135851162 the 13151942776 of 12997637966 ftp://ftp.cs.princeton.edu/pub/cs226/autocomplete/words-333333.txt
33. NLP-Spam Email Classifier using Naive Bayes - Kaggle https://www.kaggle.com/code/vrajm1209/nlp-spam-email-classifier-using-naive-bayes
34. [PDF] Proceedings of the... - ACL Anthology https://aclanthology.org/W07-14.pdf
35. AI Based Solution for Predicting Student Academic Performance and ... https://www.researchgate.net/publication/396912108_AI_Based_Solution_for_Predicting_Student_Academic_Performance_and_Real_Time_Alert_System
36. DTVM: Revolutionizing Smart Contract Execution with Determinism ... https://arxiv.org/html/2504.16552v2

37. Dictionary of Acronyms and Technical Abbreviations - Academia.edu https://www.academia.edu/64999604/Dictionary_of_Acronyms_and_Technical_Abbreviations
38. Nonhuman Witnessing - jstor https://www.jstor.org/content/pdf/oa_book_monograph/10.2307/jj.9827068.pdf
39. A trans agent of social change in incarceration - PMC - NIH https://pmc.ncbi.nlm.nih.gov/articles/PMC10108085/
40. The 2024 Joint International Conference on Computational ... https://aclanthology.org/events/lrec-2024/
41. [PDF] The Malicious Use of Artificial Intelligence - arXiv https://arxiv.org/pdf/1802.07228
42. Proceedings of the 2024 Joint International Conference on ... https://aclanthology.org/volumes/2024.lrec-main/
43. [PDF] Stick-Breaking Embedded Topic Model with Continuous ... - HAL https://hal.science/hal-05325837v1/file/2510.18786v1.pdf
44. Silk roads papers, 3rd edition - UNESCO Digital Library https://unesdoc.unesco.org/ark:/48223/pf0000396170